

AI IMPORT EXPORT

Intelligence System

Project Documentation

Developed By: SHNOOR INTERNATIONAL LLC

Technology: React.js | Vite | FastAPI | Python | PostgreSQL | OCR | Scikit-Learn

Abstract

This report documents **AI Import Export** - a full-stack import/export intelligence system developed to provide secure document handling, OCR-based text extraction, HSN classification, duty calculation, risk analysis, shipment tracking, and analytics for business workflows..

The application is built using a modern multi-layer architecture consisting of a **React.js + Vite frontend** for responsive user interaction, a **FastAPI + Python backend** for API processing and business logic, a **PostgreSQL relational database** for structured data storage, **Background processing and service orchestration** for real-time communication, and a **Python-based OCR and ML engine** trained from local datasets and model artifacts for HSN prediction and document intelligence.

The primary goal of this project is to solve practical import/export workflow challenges by delivering a modular platform that supports secure access, document-driven automation, classification, costing, tracking, and business reporting in one system.

1. Introduction

PROJECT PURPOSE

The purpose of AI Import Export is to provide an integrated operational platform for trade workflows. Unlike a basic CRUD demo, the codebase combines authentication, OCR, machine learning, business calculation, tracking, and analytics into one coordinated application.

➤ Business Problem Solved

- Import/export teams need a simple web interface to upload documents and derive useful operational data quickly.
- Trade calculations and shipment workflows often require users to switch between disconnected tools manually.
- Classification and OCR tasks can be time-consuming without automation tailored to trade-specific workflows.
- Operational systems need secure routing, persistence, and audit-friendly status tracking to be practical.

➤ Target Users

- Registered users who need secure access to document, shipment, and analytics workflows.
- Operations staff who upload trade documents and inspect OCR or classification results.
- Business users who require duty, risk, client, and shipment management tools.
- Future developers and maintainers who need a readable modular codebase.

➤ Key Value Delivered

AI Import Export provides value through an end-to-end operational workflow: account creation, secure login, file upload, OCR extraction, HSN prediction, duty calculation, risk scoring, shipment tracking, and analytics-based reporting.

2. Objectives

- Build a secure full-stack platform for import/export business operations.
- Provide document intelligence through OCR, text cleanup, and structured extraction.
- Enable HSN classification, duty calculation, and shipment risk evaluation from user inputs and extracted data.
- Provide client and shipment management workflows with transaction-aware status tracking.
- Develop a local, self-managed ML classifier using dataset-driven training and stored model artifacts.
- Support analytics summaries so shipments, payments, and risk levels can be reviewed quickly.
- Add document-based suggestions and downstream record creation from extracted OCR text.
- Allow the backend to expose protected operational data through clear REST APIs.

3. Technology Stack

Frontend

Technology	Role In Project	Evidence From Code
React 18	UI rendering, page composition, protected routes, forms, and dashboard state	<code>frontend/src/App.jsx</code> , page and component modules
Vite	Frontend dev server and production build tooling	<code>frontend/package.json</code> , <code>vite.config.js</code>

React Router DOM	Client-side routing for landing, login, register, and dashboard feature pages	<code>frontend/src/App.jsx</code>
Axios	HTTP client used for protected API communication with bearer-token support	<code>frontend/src/services/api.js</code>
Tailwind CSS	Dashboard styling and responsive layouts for feature pages	<code>frontend/src/index.css</code> , <code>frontend/src/App.css</code>
Local asset pipeline	Static images and page assets used across the dashboard experience	<code>frontend/src/assets/hero.png</code> , <code>frontend/src/assets/react.svg</code>
Custom CSS / utilities	Form layout, panels, spacing, and responsive dashboard sections	<code>frontend/src/index.css</code> , <code>frontend/src/App.css</code>

Backend

Technology	Role In Project	Evidence From Code
Python	Primary server runtime for HTTP APIs, background OCR work, and service orchestration	<code>backend/app/main.py</code>
FastAPI	REST API routing, validation, middleware registration, and request handling	<code>backend/app/main.py</code> , <code>backend/app/routers/*.py</code>
Service modules	Duty rules, OCR, HSN prediction, risk analysis, and shipment history logic	<code>backend/app/services/*.py</code>

Background tasks	Deferred OCR processing after document uploads	<code>backend/app/routers/document.py</code>
------------------	--	--

Database

Technology	Role In Project	Evidence From Code
PostgreSQL	Persistent storage for users, documents, HSN results, clients, shipments, events, and transactions	<code>backend/app/models/__init__.py</code> , <code>backend/app/core/database.py</code>
SQLAlchemy ORM	Model definitions, query handling, and relational mapping in Python	<code>backend/app/models/__init__.py</code>

Authentication

Technology	Role In Project	Evidence From Code
Passlib / password hashing	Password hashing during registration and authentication	<code>backend/app/core/security.py</code> , <code>backend/app/routers/auth.py</code>
JWTson bearer token	JWT issuance on login and protected endpoint authentication	<code>backend/app/core/security.py</code>
LocalStorage -based session persistence	Stores bearer token on the client for protected dashboard access	<code>frontend/src/App.jsx</code> , <code>frontend/src/services/api.js</code>

4. System Architecture

High-Level Architecture

ARCHITECTURE SUMMARY

The solution follows a **client-server-service** model. The React frontend is responsible for routing, dashboard UI, and form-driven workflows. The FastAPI backend provides HTTP APIs, OCR orchestration, business services, PostgreSQL connectivity, and machine learning access for HSN classification.

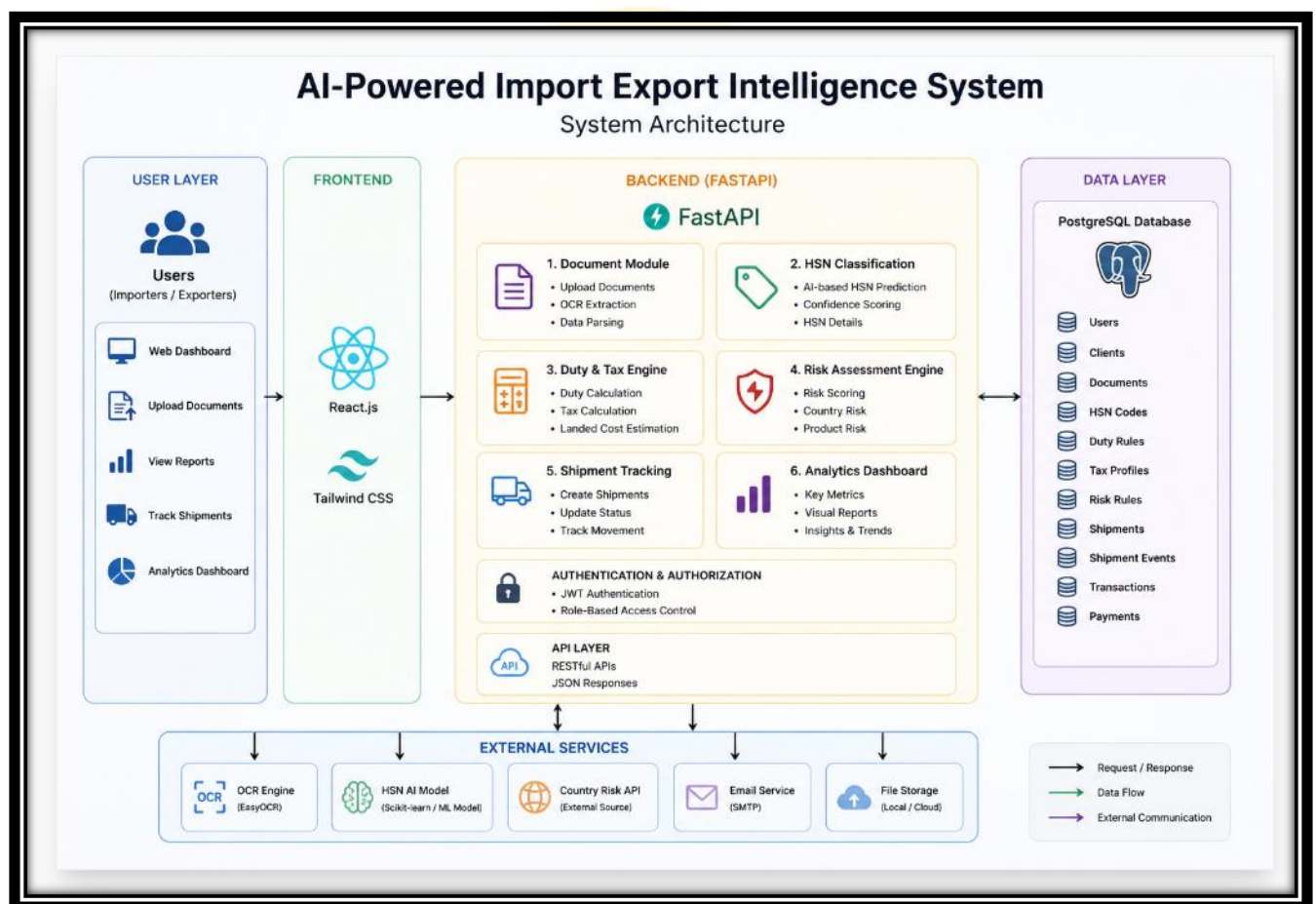


Fig : System Architecture of AI Import Export

Primary Components

- **React frontend:** pages, navigation, feature forms, analytics view, protected route behavior, API client.
- **FastAPI backend:** auth routes, document routes, HSN, duty, risk, shipment, client, and analytics endpoints.

- **Service layer:**OCR pipeline, rule resolution, classifier loading, shipment history, and client risk helpers.
- **PostgreSQL / ORM:**users, documents, HSN results, clients, shipments, events, and transactions schema foundation.
- **ML/OCR engine:**local training pipeline, OCR preprocessing, text extraction, and HSN prediction artifacts.

Data Flow

- User actions start in the browser and hit protected REST APIs for operational workflows.
- Document uploads are saved first, then OCR runs in the background while metadata remains queryable.
- HSN and duty requests use normalized user inputs and business rules or model inference to produce results.
- Shipment, transaction, and client updates persist through SQLAlchemy and feed downstream analytics summaries..

Request-Response Lifecycle

1. Frontend page issues an HTTP request based on user intent and module workflow.
2. FastAPI receives the request, validates required payload fields, and routes it to feature-specific logic.
3. Database-backed requests interact with PostgreSQL through SQLAlchemy models and session utilities.`backend/app/core/database.py` pool.
4. OCR- and ML-backed requests are handled by internal service functions and model artifacts loaded from the project.
5. Responses return as JSON to the browser, where the UI updates forms, cards, tables, and status views.
6. Background OCR tasks bypass immediate response rendering and update document status asynchronously.

5 Project Folder Structure

Folder / File	Purpose
<code>backend/app/main.py</code>	Main backend entry point, FastAPI app setup, middleware registration, router inclusion, startup cache warming.
<code>backend/app/routers/auth.py</code>	Registration and login route handlers with password hashing and JWT issuance.
<code>backend/app/routers/document.py</code>	Document upload, OCR status lookup, validation, preprocessing, and background extraction handling.
<code>backend/app/models/__init__.py</code>	Shared SQLAlchemy models for users, documents, HSN results, clients, shipments, events, and transactions.pg.
<code>backend/ml/train_hsn_model.py</code>	Model training pipeline for HSN classification artifacts.
<code>backend/app/services/hsn_classifier.py</code>	Classifier loading, validation, and inference using saved vectorizer and model files.
<code>backend/ml/sample_dataset.csv</code>	Primary curated training dataset for HSN classification examples.
<code>backend/app/services/duty_calculator.py</code>	Detailed duty and landed-cost calculation logic for friendly breakdowns.

<code>backend/app/services/duty_rules.py</code>	Rule loading, warm-cache logic, and duty-rate resolution from JSON-based trade rules.
<code>frontend/src/pages</code>	User-facing pages: landing, auth, dashboard, documents, HSN, duty, clients, risk, shipment tracking, analytics.
<code>frontend/src/services/api.js</code>	Shared Axios client with token attachment and document upload helper functions.
<code>frontend/src/components/Layout.jsx</code>	Protected dashboard wrapper and navigation shell.

6. Frontend Documentation

Frontend Pages

Page	Route	Responsibilities
Landing	/	Entry screen, project introduction, auth navigation, and route access into the protected dashboard.
Login	/login	User sign-in form, API-driven credential submission, token persistence through local storage.
Register	/register	User registration form, account creation, and protected workflow entry.
Dashboard Modules	/dashboard/*	Document upload, HSN classification, duty calculation, clients, risk engine, shipment tracking, analytics.

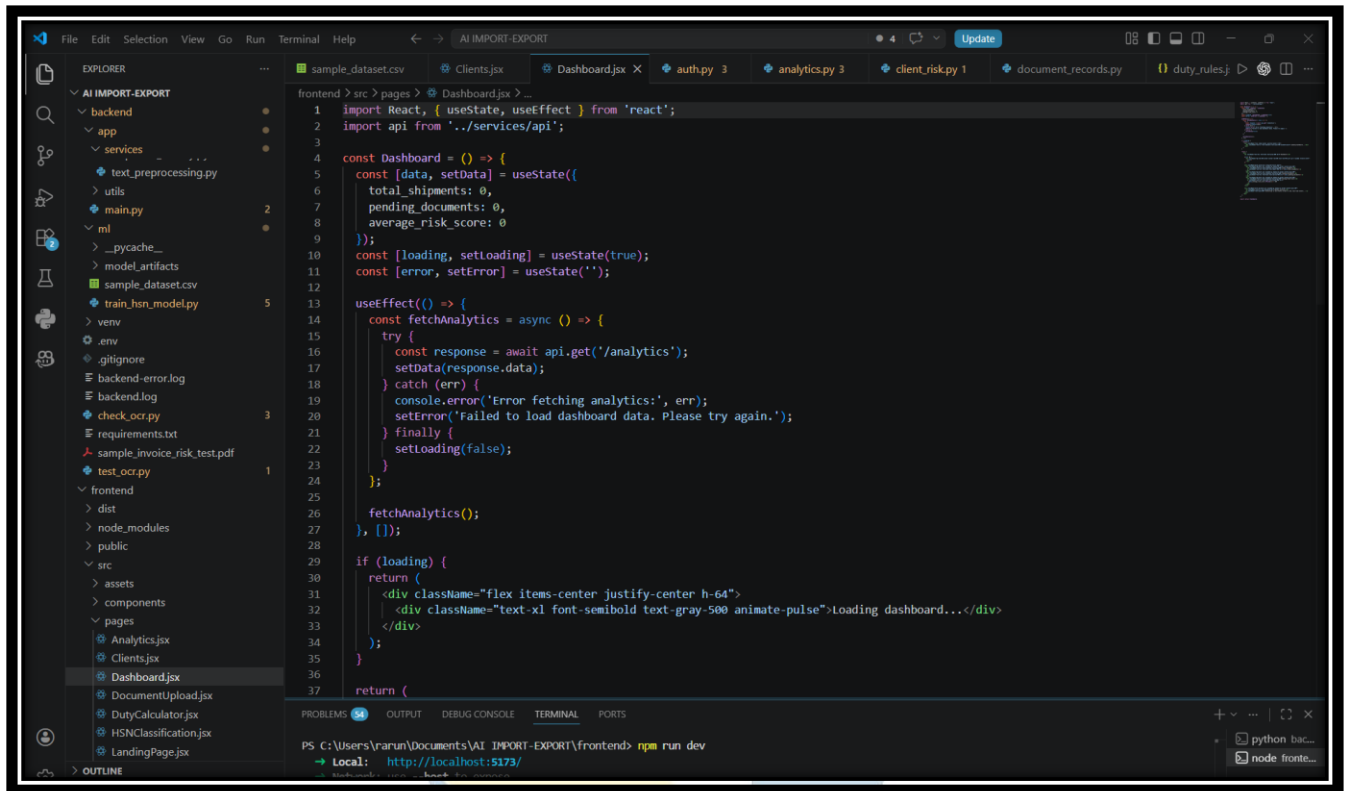


Fig : frontend files

Component and Routing Logic

- **frontend/src/App.jsx** wires the router and protected dashboard paths for all feature pages.
- **main.jsx** bootstraps the React application.
- **services/api.js** stores the shared Axios base URL and attaches the bearer token from localStorage to outgoing requests.
- **Layout.jsx** manages dashboard framing and nested route rendering for protected pages.

UI Logic and State Management

STATE MODEL

- Frontend state is lightweight and page-oriented.
- Feature-level interactions use component-local useState values.
- Protected route behavior combines token checks with route nesting to keep secured pages isolated.
- No Redux, MobX, or external state container is present.

CLIENT-SIDE BEHAVIOR

- Landing page handles introductory navigation and entry into login or register workflows.
- Document and calculator pages collect form data and send requests to backend APIs.
- Analytics and tracking pages present server-derived summaries, distributions, and operational state.
- Protected pages rely on stored token state before calling backend endpoints.

Responsive Design and Visual Style

The UI uses a structured dashboard layout with utility-driven styling, responsive sections, and distinct pages for each business workflow. The design focuses on scanability, form input clarity, and operational data presentation.

7.Backend Documentation

Server Architecture

`backend/app/main.py` `backend/app/main.py`. It initializes FastAPI, configures CORS and routing middleware, creates database tables, applies small schema maintenance helpers, warms duty rules, and serves as the API gateway for all backend feature modules.

BACKEND RESPONSIBILITIES

- Account creation and credential verification.
- Document upload and OCR status retrieval.
- HSN classification and duty calculation.
- Client and shipment management with status tracking.
- Risk analysis and analytics reporting.
- Protected API access with middleware and token-based security.

Routes and Business Logic

Module	Key Logic
<code>routers/auth.py</code>	Checks for duplicate usernames, hashes passwords, and issues JWT tokens on login.

<code>routers/document.py</code>	Validates uploads, stores file bytes, schedules background OCR, and returns document status.
<code>routers/hsn.py</code>	Runs HSN inference using the loaded vectorizer and model artifacts, then returns prediction and confidence.

8.Database Design

The relational schema is declared in `backend/app/models/__init__.py` and contains seven primary entity groups. The schema is operational and readable, supporting user identity, document persistence, HSN classification outputs, clients, shipments, shipment event history, and transaction-based risk context.

Table	Purpose	Important Fields
<code>users</code>	Stores registered user accounts.	<code>id</code> , <code>username</code> , <code>email</code> , <code>hashed_password</code> , <code>role</code> , <code>is_active</code>
<code>documents</code>	Stores uploaded files and OCR outputs.	<code>id</code> , <code>filename</code> , <code>content_type</code> , <code>file_size</code> , <code>raw_text</code> , <code>cleaned_text</code> , <code>extracted_text</code> , <code>ocr_status</code> , <code>owner_id</code>
<code>hsn_results</code>	Stores business clients and route metadata.	<code>id</code> , <code>predicted_hsn</code> , <code>confidence_score</code> , <code>document_id</code>
<code>clients</code>	Stores chatbot feedback events, rewards, predicted/corrected intent, and correction history.	<code>id</code> , <code>name</code> , <code>from_country</code> , <code>to_country</code> , <code>created_at</code>
<code>shipments</code>	Stores shipment records and tracking state.	<code>id</code> , <code>tracking_number</code> , <code>status</code> , <code>current_status</code> , <code>risk_score</code> , <code>client_id</code> , <code>document_id</code> , <code>owner_id</code>

shipment_events	Stores shipment history events.	id, shipment_id, status, timestamp
transactions	Stores payment and risk-related transaction records.	id, client_id, shipment_id, amount, status, timestamp

Relationships

- `documents.owner_id` references `users.id` and links uploaded files to authenticated users.
- `hsn_results.document_id` references `documents.id` and keeps each classification tied to one processed document.
- `shipments.owner_id` references `users.id` to separate shipment ownership by authenticated user.
- `shipments.document_id` references `documents.id` to associate shipment workflows with uploaded source documents.
- `shipment_events.shipment_id` references `shipments.id` for time-ordered tracking history.
- `transactions.shipment_id` references `shipments.id` to connect payment state with shipment risk context.

Non-Relational Supporting Data

CSV / JSON DATA ASSETS USED BY THE PROJECT

Asset	Purpose
backend/ml/sample_dataset.csv	Stores HSN training rows used by the ML pipeline.
backend/app/data/duty_rules.json	Stores country and product-code based duty configuration rules.

Asset	Purpose
backend/ml/model_artifacts/hsn_model.joblib	Persists the trained classifier artifact.
backend/ml/model_artifacts/hsn_vectorizer.joblib	Persists the trained TF-IDF vectorizer artifact.

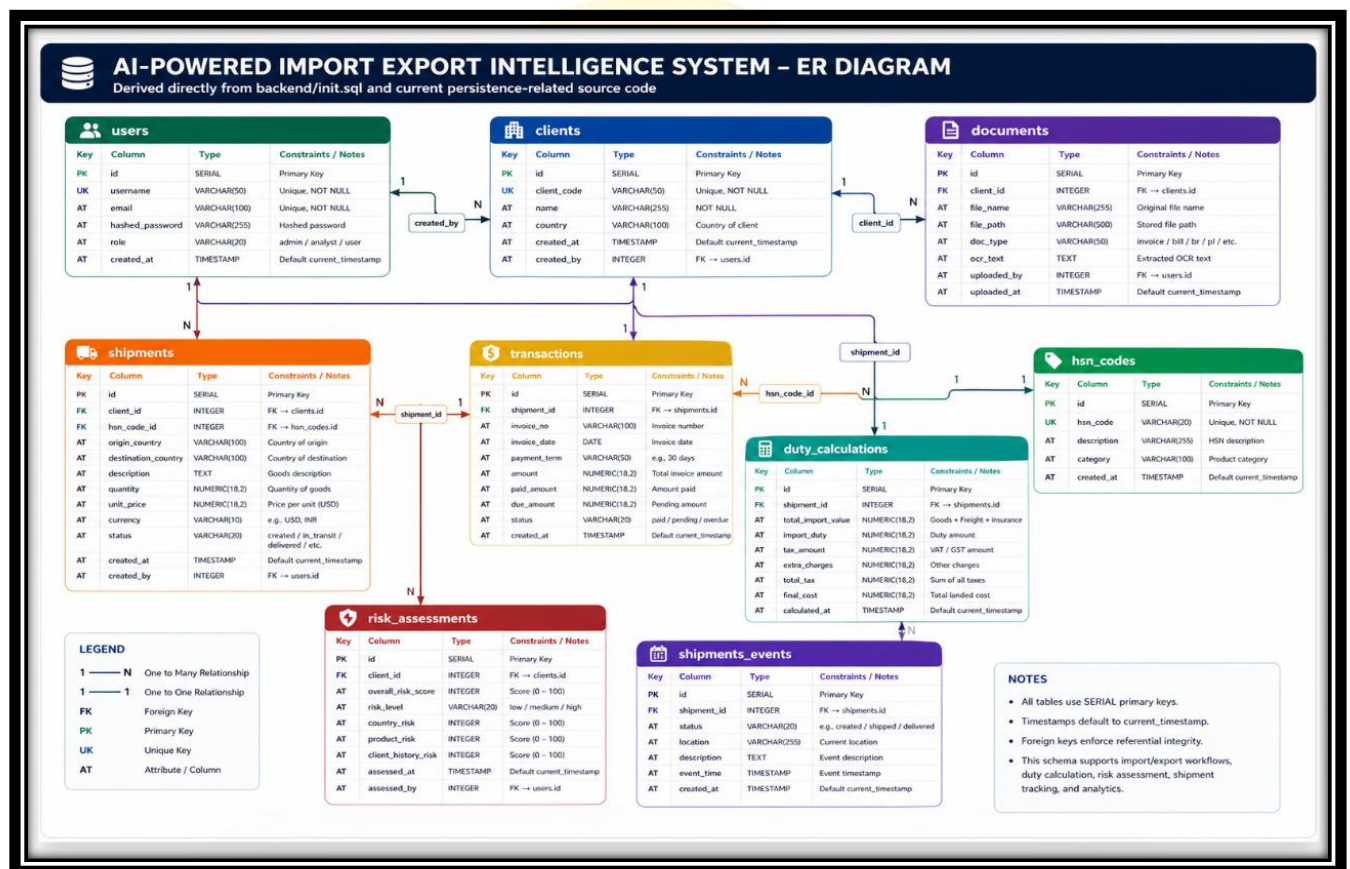


Fig : ER Diagram for Database Design

9. API Documentation

Method	Endpoint	Purpose
POST	/api/auth/register	Create a new user.
POST	/api/auth/login	Authenticate and receive JWT token.
POST	/api/documents	Upload document for OCR.
GET	/api/documents/{document_id}	Fetch OCR status and extracted text.
POST	/api/hsn	Predict HSN code.
POST	/api/duty	Calculate simple duty.
POST	/api/duty/friendly	Calculate detailed duty breakdown.
POST/GET/PUT/DELETE	/api/clients	Manage clients and transactions.

Internal Service APIs

Type	Name	Purpose	Exposure
Function	<code>perform_ocr / process_document_ocr</code>	Extract text from PDFs and images, clean it, and persist document state.	Internal local service

Function	<code>predict_hsn</code>	Run normalized-text inference with trained vectorizer and classifier.	Internal local service
Function	<code>calculate_friendly_duty_breakdown</code>	Compute detailed landed cost summary from pricing and route values.	Internal local service
Function	<code>analyze_client_risk / analyze_shipment_risk</code>	Evaluate operational risk using business route and transaction context.	Internal local service

10. Modules & Features

10.1 Landing Page Module

- The landing page introduces the AI Import Export Intelligence System to the user.
- It acts as the public entry point before authentication.
- Users can navigate to login or registration from this page.
- It provides the first overview of the platform and its operational purposes.



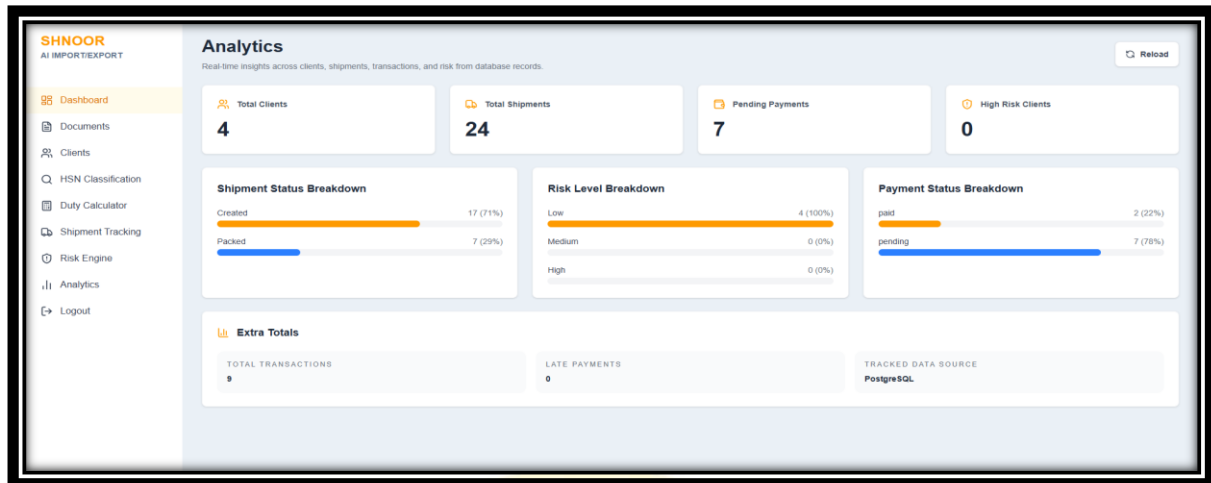
10.2 Authentication Module

- Users can create a new account using the registration page.
- Existing users can log in using their email and password.
- Passwords are securely hashed before storage in the database.
- JWT-based access tokens are generated after successful login.
- Protected routes are accessible only to authenticated users.
- Frontend authentication state is maintained using localStorage.

The image displays two side-by-side screenshots of the application's authentication interface. The left screenshot, titled 'Login to System', features a white card with a light blue border. It contains an 'Email' input field with the text 'arun', a 'Password' input field with four dots, an orange 'Sign In' button, and a link that says 'Don't have an account? Register'. The right screenshot, titled 'Register Account', also features a white card with a light blue border. It contains a 'Username' input field with the text 'arun', an empty 'Email' input field, a 'Password' input field with four dots, an orange 'Register' button, and a link that says 'Already have an account? Login'. Both cards are set against a light gray background with a large, faint 'SHNOOR INTERNATIONAL' watermark.

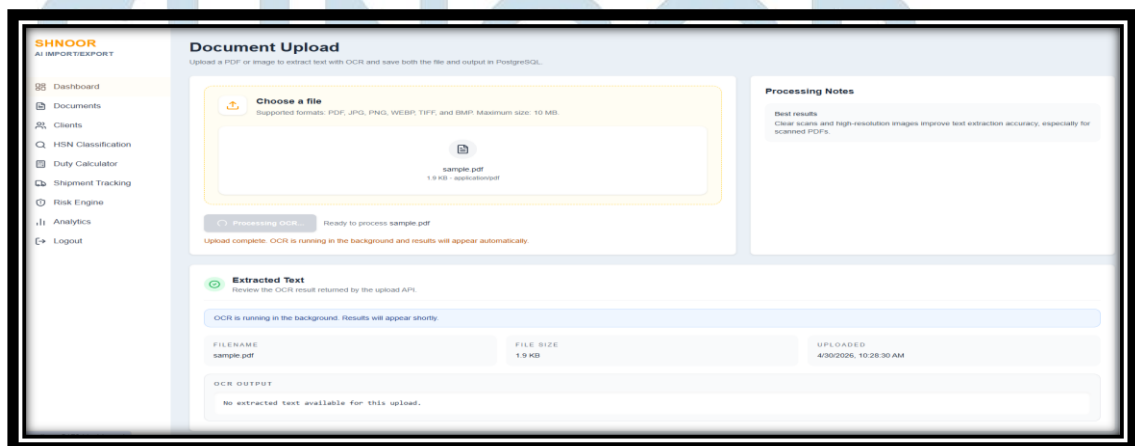
10.3 Dashboard Module

- The dashboard acts as the main control center after login.
- It gives users access to all major workflows in one protected interface.
- It helps users navigate to document upload, HSN prediction, duty calculation, risk analysis, clients, shipment tracking, and analytics.
- Dashboard cards and summary panels provide a quick operational overview.



10.4 Document Upload and OCR Module

- Users can upload PDF and image-based trade documents.
- The system validates file type and size before processing.
- OCR runs in the background after upload.
- Extracted text is cleaned and normalized for downstream use.
- OCR status, extracted text, filename, file size, and upload metadata are stored in the database.
- The processed document can later support client suggestions and shipment-related workflows.



10.5 HSN Classification Module

- Users can enter a product description and receive a predicted HSN code.
- The prediction system uses a trained TF-IDF vectorizer and machine learning classifier.
- Prediction confidence is also returned along with the HSN result.
- The model is trained using the sample_dataset.csv dataset.

- This module supports trade classification workflows and downstream customs-related calculations.

SHNOOR
AI IMPORT/EXPORT

HSN Classification
Enter a product description to predict the most likely HSN code using the trained text model.

Describe the product
Include material, usage, or type so the classifier can choose the right HSN category.

Product Description
grey shirt

Example inputs: cotton shirt, medical syringe, electronic circuit board, leather handbag

Predict HSN Code Ready to classify the current description.

Classification Notes

Best results
Add product type, material, and intended use. More specific descriptions improve category matching.

Model behavior
Confidence is a relative score from the classifier. Lower confidence usually means the input is broad or ambiguous.

Classification Result
Review the HSN prediction returned by the API.

PREDICTED HSN	CONFIDENCE	INPUT LENGTH
6205	41.73%	2 words

SUBMITTED DESCRIPTION
grey shirt

10.6 Duty Calculator Module

- Users can calculate import duty using country of origin, HSN code, and product value.
- The system uses JSON-based duty rules stored in duty_rules.json.
- A simple duty calculation endpoint returns duty percentage and calculated duty.
- A friendly duty breakdown endpoint provides landed-cost style output.
- Repeated calculations can be accelerated through cached results.
- This module supports customs estimation and cost planning.

SHNOOR
AI IMPORT/EXPORT

Duty Calculator
Estimate total import cost using origin, destination, product code, and related shipment charges.

Enter duty details
Provide the shipment route, product code, and price inputs to calculate import duty, tax, and final landed cost.

From Country: India To Country: United States

Product Code (HSN): 8708 Product Price: 150

Shipping Cost (Optional): 10 Insurance Cost (Optional): e.g. 5

Total import value is calculated as product price plus shipping and insurance, then duty and tax are applied for the destination country.

Calculate Duty Ready to estimate landed cost for this shipment.

Calculation Notes

Breakdown logic
The calculator adds shipping and insurance to the product price, applies import duty, then tax, and then any extra charges.

Best results
Use the product code from HSN classification and enter accurate shipping costs for a more reliable estimate.

Calculation Result
Review the full duty, tax, and final cost breakdown returned by the API.

IMPORT VALUE	IMPORT DUTY	TAX	EXTRA CHARGES	TOTAL TAX	FINAL COST
\$160.00	\$12.00	\$0.00	\$1.60	\$13.60	\$183.60

FROM COUNTRY IN **TO COUNTRY** US **PRODUCT CODE** 8708

10.7 Client Management Module

- Users can create, view, update, and delete client records.
- Each client record stores name, source country, and destination country.

- Client transaction summaries are linked with the client profile.
- OCR-extracted document data can provide suggestions for new client entries.
- Client risk summaries can be retrieved from the backend.
- This module supports business relationship management within the trade workflow.

SHNOOR
AI IMPORT/EXPORT

Client Risk Management
Manage clients, search by client ID, and review payment-risk details from real transaction history.

Add new client
Store client name and route details, then use the View Risk action to combine client info with payment behavior.

Name: From Country: To Country:

Client name: IN US

Client Search
Search by Client ID
Q. Type an id like 3

How to use
Create or update clients here, then use View Risk to fetch their current payment-risk profile from transaction data.

Clients
Search, edit, delete, or inspect risk for any client in the system.

ID	NAME	FROM	TO	TRANSACTIONS	LATE	PENDING	ACTIONS
11	provided software and devices must be aligned with project purposes only.	IN	US	1	0	1	Edit Delete View Risk
10	Client 2	IN	US	6	0	6	Edit Delete View Risk
9	Sai	IN	US	1	0	1	Edit Delete View Risk
3	arun	IN	US	2	0	0	Edit Delete View Risk

10.8 Risk Analysis Module

- Users can evaluate shipment risk using route, product code, declared price, and tax values.
- The backend calculates shipment risk using business rules and contextual logic.
- Client-specific risk summaries can also be generated.
- Risk events are published through the event pipeline for operational traceability.
- This module supports compliance awareness and exception monitoring.

SHNOOR
AI IMPORT/EXPORT

Risk Engine
Evaluate shipment risk and client payment behavior from the same workspace.

Shipment Risk
Score customs and trade risk using route, product code, shipment value, and total tax.

From Country: To Country:

China United States

Product Code (HSN): Product Price:

e.g. 8708 e.g. 7000

Total Tax:

e.g. 1200

Higher shipment value, higher tax, unfamiliar product codes, and risky route combinations all raise the score.

Client Payment Risk
Look up payment behavior risk from historical client transaction data in PostgreSQL.

Client ID:

e.g. 1

This checks transaction count, late payments, pending payments, shipment value history, and payment timing pattern.

Risk Notes

Shipment scoring
Uses route, code familiarity, shipment value, and tax pressure to produce a 0 to 100 score.

Client scoring
Uses historical transactions, late payments, pending invoices, high values, and irregular payment timing.

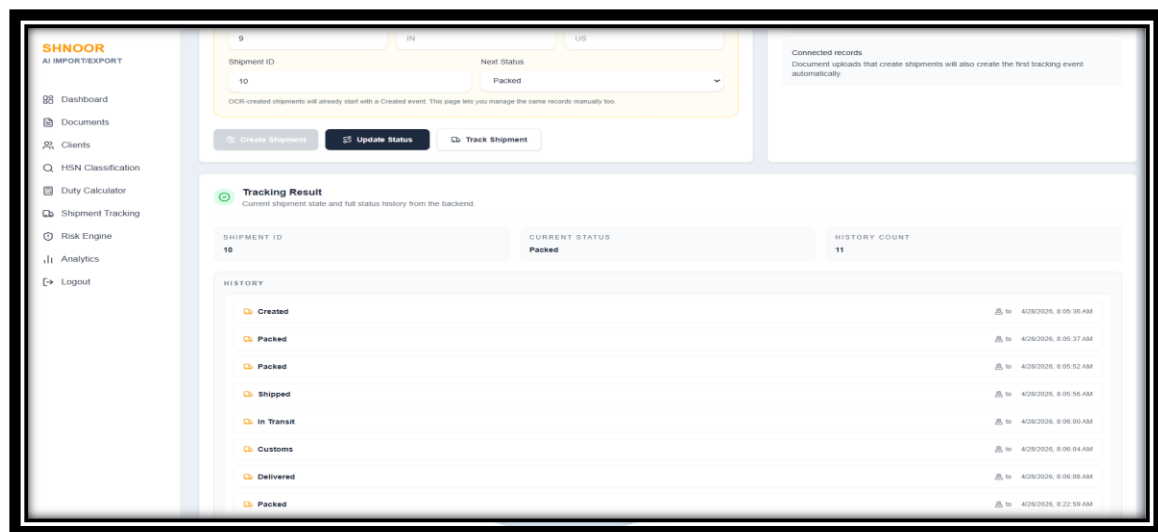
Client Notes

Needs transaction data
This section returns results only when the selected client already has payment history records in the database.

What you will get
A client summary with total transactions, late payments, pending payments, plus a payment behavior risk level.

10.9 Shipment Tracking Module

- Users can create new shipment records from the dashboard.
- Unique tracking numbers are generated automatically for shipments.
- Shipment status can be updated across predefined tracking stages.
- Each shipment status change is stored as a shipment event.
- Users can retrieve shipment history to review the complete lifecycle of a shipment.
- Shipment ownership is linked to the authenticated user.



10.10 Analytics Module

- The analytics module provides summary information about operational records.
- It returns totals such as total clients, total shipments, total transactions, pending payments, late payments, and high-risk clients.
- Distribution endpoints group shipment statuses, payment statuses, and risk levels.
- Dashboard analytics also include total shipments, pending document-related values, and average risk score.
- These outputs help users understand current business performance and workflow status.

15. Workflow Explanation

End-to-End User Flow

1. User lands on the home page of the AI Import Export system.
2. User signs in with existing credentials or creates a new account.
3. After authentication, the user enters the protected dashboard.
4. The user selects a workflow such as document upload, HSN classification, duty calculation, client management, risk analysis, shipment tracking, or analytics.
5. If a document is uploaded, the backend stores it and starts OCR processing in the background.
6. Extracted document text is cleaned and stored for later use.
7. The user can classify products using the HSN prediction module.
8. The user can calculate customs duty and landed-cost related values using duty rules.
9. The user can create clients and manage their transaction-related summaries.
10. The user can create shipments, update shipment status, and review tracking history.
11. The user can run risk analysis for shipment or client-based scenarios.
12. Analytics endpoints summarize operational data for dashboard reporting.
13. Protected APIs ensure that only authenticated users can access secured records and workflows.

21. Conclusion

AI Import Export is a well-integrated full-stack application designed to support modern trade and logistics workflows through automation and intelligent processing. The project combines React frontend development, FastAPI backend services, SQLAlchemy-based persistence, OCR-based document extraction, machine learning-driven HSN classification, duty calculation, shipment tracking, client handling, and analytics in a single operational platform.

From an engineering perspective, the project demonstrates a strong academic and practical implementation of an AI-supported import-export intelligence system. It is suitable as a major project, portfolio project, or functional MVP. Future improvements can focus on broader testing coverage, production deployment hardening, stronger infrastructure integrations, and more advanced monitoring or reporting capabilities.